

Brasil 2026

Criando multiplos agentes usando Agent Framework Framework + Ai Foundry

Global Azure 2026 | Community for Community

Márcio R. Nizzola

4x MVP Microsoft - Developers
Technologies

Principal Architect na CI&T

Professor na Etec de Itu

Fundador do Meetup Itu Developers

www.linkedin.com/in/nizzola

marcionizzola.medium.com

<https://linktree.com/nizzola>

@nizzola.dev

Estudando programação a partir de 1989

Vivendo de software desde 1992

Principais tecnologias: C#, Python, IA,
Javascript, Angular, React, Azure, Aws, Php,
Vb



www.nizzola.com.br

Comunidade



Membro fundador

Comunidade com 5 anos já organizou 12 Meetups,
Palestras e Lives (TechChat).

ItuDevelopers®



www.nizzola.com.br

What is Azure AI Foundry?

Build AI in 2025



A



Azure AI Foundry

Foundry Models

Foundry Agent Service

Azure AI
Search

Azure AI
Services

Azure
Machine Learning

Azure AI
Content Safety

Foundry Observability

Security & Governance



Control



Azure Kubernetes Service



Azure App Service



Azure Container Apps



Azure Functions



Serverless



Copilot
Studio



Visual
Studio



GitHub



Foundry
SDK



Cloud



Azure



Azure Arc



Foundry
Local

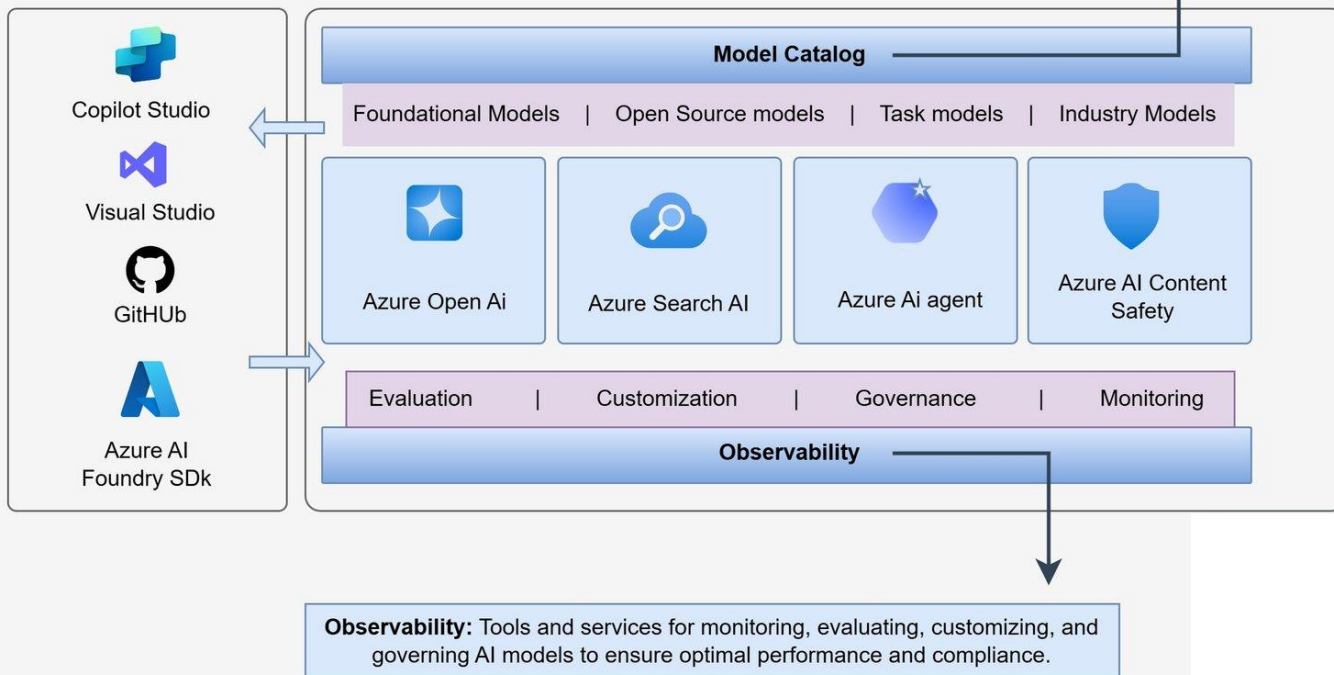


Edge

Model Catalog: Central repository for storing and managing various AI models, including foundational, open-source, task-specific, and industry-specific models.

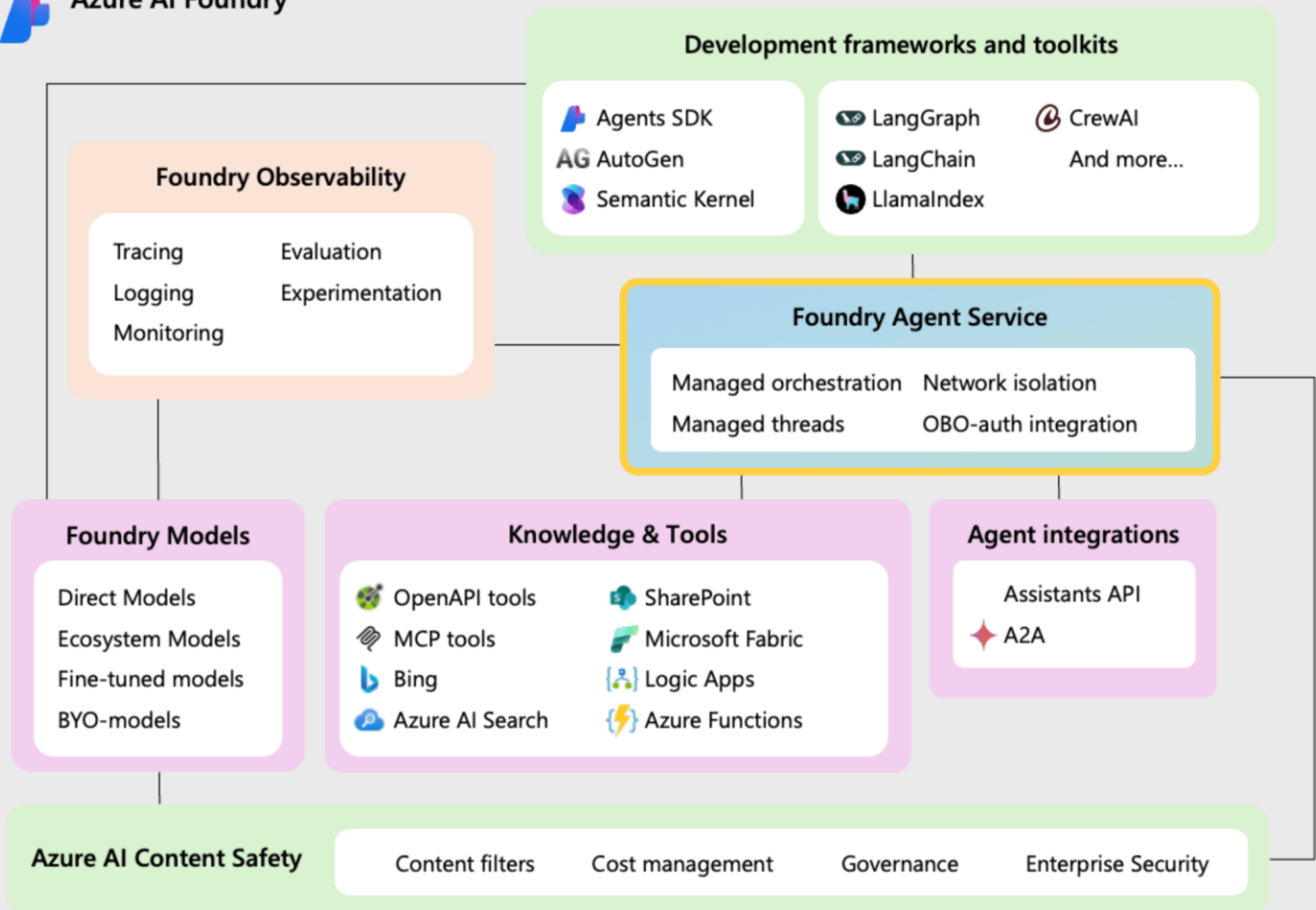


Azure AI Foundry





Azure AI Foundry



Entrando no Portal do Foundry

portal.azure.com



Microsoft Agent Framework Framework

A Evolução Unificada de Semantic Kernel e AutoGen



Global Azure 2026



By Community · For Community



Azure AI Agents

Por Que um Novo Framework?

Semantic Kernel

Construído originalmente para integrar LLMs em apps convencionais (Kernel/Plugins). O conceito de conceito de "agente" foi adicionado depois (retrofitted) sobre primitivas de orquestração.

AutoGen

Focado em pesquisa e inovação em conversas multi-agente dinâmicas, mas com menos foco em estabilidade empresarial e integração de sistemas legados.

Agent Framework

Nasce "Agent-First": projetado desde o primeiro dia para ser a base de agentes inteligentes, workflows estruturados e execução de nível empresarial.

O Que é o Microsoft Agent Framework?

A NOVA BASE UNIFICADA

- O **Microsoft Agent Framework (AF)** é o sucessor direto do direto do Semantic Kernel e do AutoGen.
- Criado pelas mesmas equipes para consolidar o aprendizado em orquestração de IA.
- Oferece um modelo de programação unificado para para **.NET** e **Python**.

FOCO EMPRESARIAL

-  Projetado para aplicações de IA de **nível empresarial, empresarial**, robustas e seguras.
-  Integração profunda com o ecossistema Azure e Microsoft Foundry.
-  Foco em estabilidade, telemetria e gerenciamento de gerenciamento de estado avançado.

Comparação de Ecosystems

CARACTERÍSTICA	SEMANTIC KERNEL	AUTOGEN	AGENT FRAMEWORK
Design de Agentes	Adicionado depois (Retrofitted)	Sim (Foco em Conversa)	✓ Nativo (Agent-First)
Workflows	Orquestração Manual	Conversacional Dinâmico	✓ Baseado em Grafos
Estado e Sessão	Robusto (Kernel-based)	Básico / Volátil	✓ Avançado e Persistente
Linguagens	.NET / Python / Java	Python (Principal)	✓ Unificado .NET / Python
Foco Principal	Integração de LLM	Pesquisa Multi-Agente	✓ Aplicações Enterprise

Os Dois Pilares do Framework



Agentes (Agents)

Entidades individuais que utilizam LLMs para processar entradas, raciocinar sobre tarefas e executar ações.

- ✓ Suporte a **múltiplos provedores** (Azure, OpenAI, Ollama)
- ✓ Uso de ferramentas e servidores **MCP**
- ✓ Conversas multi-turno e memória contextual
- ✓ Raciocínio dinâmico e autônomo



Workflows

Estruturas baseadas em grafos que conectam agentes e funções para orquestrar tarefas complexas e multi-etapas.

- ✓ Roteamento **Type-Safe** determinístico
- ✓ Sistema de **Checkpointing** e persistência
- ✓ Suporte nativo a **Human-in-the-loop**
- ✓ Execução sequencial, paralela ou condicional

Vantagens para o Desenvolvedor Enterprise

TYPE SAFETY

Redução drástica de erros em tempo de execução através de **tipagem tipagem forte** em .NET e Python, garantindo contratos claros entre agentes e ferramentas.

GESTÃO DE SESSÃO

Estado persistente nativo para **conversas de longa duração**. Gerencie o histórico e o contexto de múltiplos usuários de forma escalável e segura.

MIDDLEWARE

Intercepte e modifique ações do agente em tempo real. Essencial para implementar **filtros de segurança**, auditoria e lógica de negócios customizada.

TELEMETRIA

Monitoramento detalhado de performance, latência e **custos de de tokens**. Integração nativa com ferramentas de observabilidade observabilidade enterprise.

Workflows em Grafo: O Diferencial Estrutural

CONTROLE DE EXECUÇÃO



Orquestração Baseada em Grafos

Permite definir caminhos de execução **explícitos e determinísticos** entre múltiplos agentes e funções.



Determinismo vs. Autonomia

Misture lógica de programação clássica (if/else, loops) com o raciocínio dinâmico e probabilístico da IA.

RESILIÊNCIA E INTERAÇÃO



Human-in-the-loop

Suporte nativo para pausar a execução, solicitar aprovação aprovação ou entrada humana e retomar o processo.



Checkpointing e Persistência

Capacidade de salvar o estado do workflow em qualquer ponto, permitindo **recuperação de falhas** e processos de longa duração.

Integração Nativa com MCP

Model Context Protocol

O QUE É O MCP?

-  Padrão aberto criado pela **Anthropic** e adotado pela **Microsoft** para conectar LLMs a dados e ferramentas.
-  Elimina a necessidade de escrever conectores customizados para cada nova ferramenta ou API.
-  Promove a **interoperabilidade** total entre diferentes frameworks de agentes.

BENEFÍCIOS NO FRAMEWORK

-  **Tool Integration:** Conecte seus agentes a bancos de dados, sistemas de arquivos e APIs externas instantaneamente.
-  Reutilize o mesmo "Tool" em diferentes agentes e até e até em outros frameworks compatíveis com MCP.
-  Segurança padronizada para o acesso a contextos e contextos e execução de funções externas.

Quando Usar Agentes vs Workflows?



Use Agentes Quando:

- ✓ A tarefa exige **raciocínio dinâmico** e tomada de decisão em tempo real.
- ✓ O caminho para a solução não é estritamente definido ou linear.
- ✓ Há necessidade de conversas multi-turno complexas com o usuário.
- ✓ O agente precisa escolher autonomamente quais ferramentas usar.



Use Workflows Quando:

- ✓ O processo tem **etapas claras**, sequenciais ou condicionais.
- ✓ É necessário controle rígido e determinístico sobre a execução.
- ✓ A tarefa exige **checkpoints de segurança** ou aprovação humana.
- ✓ Você precisa de roteamento type-safe entre diferentes agentes.



Regra de Ouro: Se você pode resolver o problema com uma função de código simples e determinística, faça isso em vez de usar um agente de IA.

Mão na Massa: Exemplo em C# (.NET)

```
$ dotnet add package Microsoft.Agents.AI.Foundry --prerelease
```

PROGRAM.CS

```
using Microsoft.Agents.AI;

// Inicializa o cliente e o agente
AIAgent agent = new AIProjectClient(projectUri, credential)
    .AsAIAgent(
        model: "gpt-4o",
        instructions: "Você é um assistente técnico breve."
    );

// Executa uma interação
var response = await agent.RunAsync("Como o AF substitui o SK?");

Console.WriteLine(response);
```

O Futuro e a Comunidade Open Source

COMPROMISSO ABERTO

-  O Microsoft Agent Framework é **totalmente Open Source**, seguindo o legado de sucesso do Semantic Kernel e AutoGen.
-  Desenvolvido em colaboração direta com a comunidade global de desenvolvedores de IA.
-  Evolução contínua com atualizações frequentes e suporte a novos modelos de linguagem.

CAMINHO DE MIGRAÇÃO

-  A Microsoft fornece **guias de migração detalhados** para **detalhados** para quem já utiliza os frameworks anteriores.
-
-  **Semantic Kernel**: Mapeamento de Kernels e Plugins para Plugins para Agentes e Tools.
 -  **AutoGen**: Transição de AssistantAgents para o novo novo modelo unificado.

Orquestração Multi-Agente: Padrões



SEQUENCIAL

Fluxo linear onde o resultado de um agente alimenta o próximo. Ideal para pipelines de processamento de dados.



CONCORRENTE

Múltiplos agentes trabalham em paralelo para resolver sub-tarefas. Um agente "Orquestrador" consolida a resposta final.



HANDOFF

Transferência dinâmica de tarefas entre agentes especializados (ex: Agente de Suporte passa para Agente de Vendas).

Vantagem: Agentes menores e especializados são mais precisos, baratos e fáceis de manter do que um único agente monolítico.

Anatomia de um Workflow (Graph-based)



EXECUTORS

Unidades de processamento do grafo. Podem ser **Agentes de IA** ou funções de código C# determinísticas.



EDGES

Conexões que definem o **fluxo de mensagens** entre os Executors. Definem quem fala com quem.



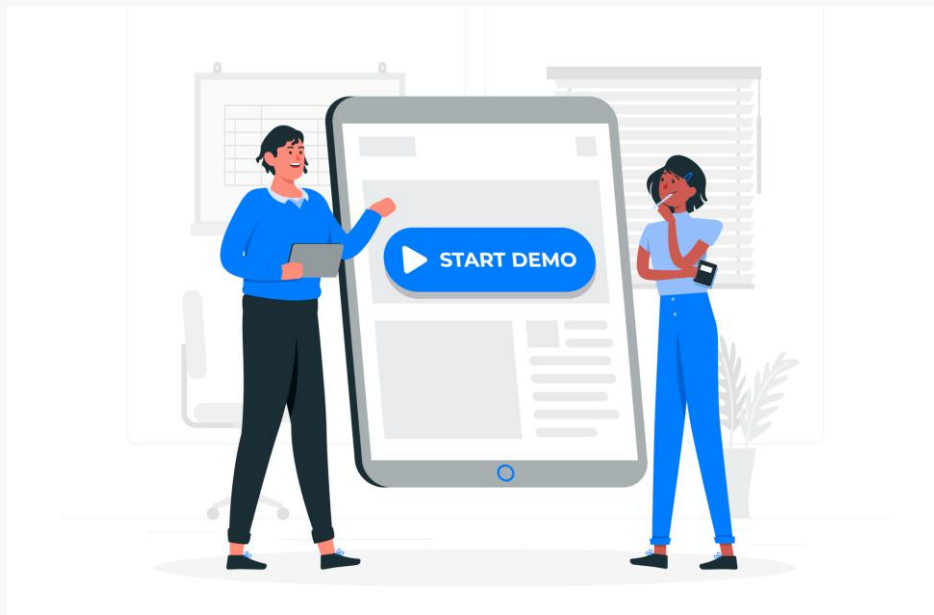
ROTEAMENTO

Lógica condicional para decidir o próximo passo com base no **conteúdo da mensagem** ou estado do workflow.



CHECKPOINTING

Persistência do estado em cada nó. Permite **resiliência**, **auditoria** e **auditoria** e interações humanas (Human-in-the-loop).



<https://ai.azure.com/>

Vai nessa, mostra pra eles, vai funcionar tudinho.....



Projeto com Funções (Local FunctionTool)

O QUE É?

Transforma qualquer **método C#** em uma ferramenta que o agente pode chamar.

QUANDO USAR?

Acesso a recursos locais, cálculos determinísticos rápidos e rápidos e integração com código legado no mesmo processo.

```
// 1. Defina sua lógica local
[Description("Calcula o imposto sobre um valor")]
public double CalcularImposto(double valor) => valor * 0.15;

// 2. Crie a ferramenta
var tool = AIAgentFunctionTool.Create(CalcularImposto);

// 3. Injete no agente
var agent = new AIAgent(...);
agent.AddTool(tool);

// 4. O agente decide quando chamar!
var res = await agent.RunAsync("Qual o imposto para 1000?");
```


Projeto com MCP (Remote Tools)

O QUE É?

Consome ferramentas expostas por um servidor **MCP** **MCP externo** (via stdio, http ou websocket).

QUANDO USAR?

Ferramentas compartilhadas entre múltiplos agentes, APIs de terceiros e acesso a dados em ambientes isolados.

```
// 1. Conecte ao servidor MCP externo
var transport = new StdioMcpTransport("npx", "-y
@modelcontextprotocol/server-github");
var mcpClient = await McpClientFactory.CreateAsync(transport);

// 2. Liste e injete as ferramentas
var mcpTools = await mcpClient.ListToolsAsync();
agent.AddTools(mcpTools.Cast<AITool>());

// 3. O agente descobre e usa dinamicamente!
var res = await agent.RunAsync("Quais os últimos commits do repo X?");
```

Workflows vs. Agentes: Guia Prático

CRITÉRIO	AGENTE (DINÂMICO)	WORKFLOW (DETERMINÍSTICO)
Fluxo de Execução	Definido pelo LLM em tempo real	Definido pelo Desenvolvedor no grafo
Previsibilidade	Variável (Probabilístico)	Alta (Determinístico)
Complexidade	Alta (Raciocínio complexo)	Média (Processo estruturado)
Melhor para...	Criatividade, Exploração, Chat	Compliance, Processos de Negócio

The Microsoft Azure AI Portfolio

Azure AI Studio

The place to test, build and deploy AI solutions

Azure AI Services

Pre-built models, APIs and SDKs to infuse into custom apps



Azure AI
Vision



Azure AI
Speech



Azure AI
Language



Azure AI
Translator

Azure Open AI Services

AI Services complements LLM workloads



Azure OpenAI
Service



Azure AI
Search



Azure AI Document
Intelligence



Azure Content Safety

Azure Machine Learning

Advanced tools for designing and fine-tuning specialized AI models

Machine Learning
Studio



Model
Catalog



Prompt
Flow



Responsible AI

MLOps
And LLMOps

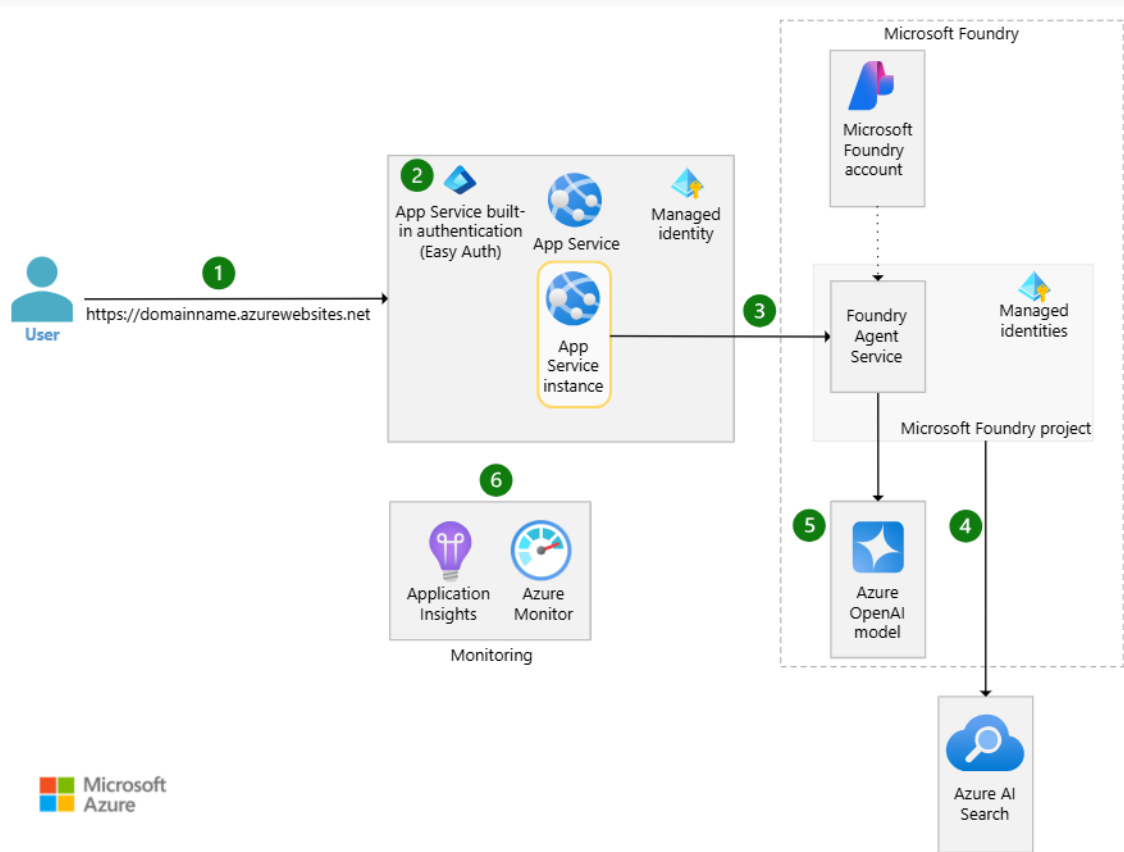
Custom Copilots

Prompt
Engineering

Azure AI Scenarios

Technical Capabilities to enable portfolio

Arquiteturas de Referência



[Centro de Arquitetura do Azure - Azure Architecture Center | Microsoft Learn](#)

[Basic Microsoft Foundry Chat Reference Architecture - Azure Architecture Center | Microsoft Learn](#)

Microsoft's AI Ecosystem



Copilot



GitHub



Azure



Sales



Marketing



Customer Service



Finance



Supply Chain
Management



Security



Bing



Copilot Studio



Word



Excel



PowerPoint



Outlook



Teams



Loop



Viva



Planner



OneNote



Forms



Whiteboard



OpenAI
Model Family



DeepSeek
Model Family



Phi SLM
Model Family



Mistral AI
Model Family



Meta Llama
Model Family



Jais G42
Model Family



Cohere
Model Family



Databricks
Model Family



Hugging Face
Model Family



Search
AI



Text
AI



Document
AI



Vision
AI



Language
AI



Speech
AI



Video
AI



Prompt
Shields



Protected
Material



Groundness
Detection



Content
Safety



Prompt
Flow



Machine
Learning
Studio



Data
Preparation



Data
Labeling



Model Catalog
+1600 Models



AutoML



Experiments



Model
Training



Model
Registry

Conclusão e Próximos Passos



Unificação

O melhor do **Semantic Kernel** e **AutoGen** em um único framework nativo.



Enterprise-Ready

Foco total em **segurança, estado e telemetria** para produção.



Controle Total

Workflows em grafo para orquestração complexa e determinística.

Comece a construir hoje mesmo:

learn.microsoft.com/en-us/agent-framework

Obrigado pela atenção! Perguntas?

Referências e Recursos Adicionais



Documentação Oficial do Microsoft Agent Framework

learn.microsoft.com/en-us/agent-framework/



Repositório de Exemplos (Agent Framework Samples)

github.com/microsoft/Agent-Framework-Samples

github.com/microsoft/agent-framework/tree/main/dotnet/samples



Blog de Engenharia da Microsoft (Azure AI)

devblogs.microsoft.com/agent-framework/



Model Context Protocol (MCP) Specification

modelcontextprotocol.io

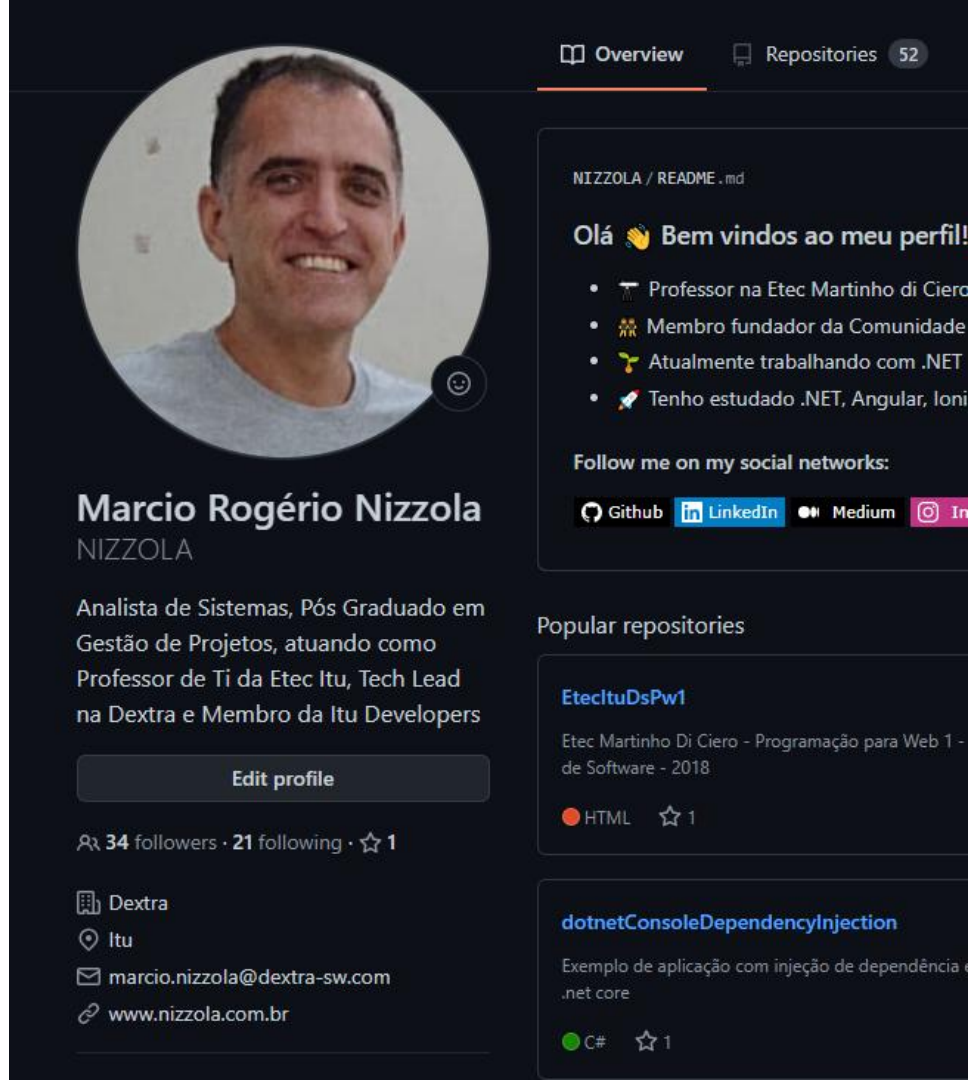
Contatos

<https://linktr.ee/NIZZOLA>

<https://github.com/nizzola>

<https://www.linkedin.com/in/nizzola>

Mais de 100 artigos sobre .NET
marcionizzola.medium.com



The screenshot shows a GitHub profile for Marcio Rogério Nizzola. At the top, there are tabs for 'Overview' (selected) and 'Repositories' (52). The profile picture is a circular headshot of a smiling man. Below the picture is the name 'Marcio Rogério Nizzola' and the username 'NIZZOLA'. A bio describes him as a Systems Analyst, Graduate in Project Management, Professor at Etec Itu, Tech Lead at Dextra, and a member of the Itu Developers community. There is an 'Edit profile' button. Below the bio, it shows '34 followers' and '21 following'. A list of links includes 'Dextra', 'Itu', 'marcio.nizzola@dextra-sw.com', and 'www.nizzola.com.br'. On the right side, there is a section titled 'NIZZOLA / README .md' with a greeting 'Olá 👋 Bem vindos ao meu perfil!' and a list of bullet points: 'Professor na Etec Martinho di Ciero', 'Membro fundador da Comunidade', 'Atualmente trabalhando com .NET', and 'Tenho estudado .NET, Angular, Ioni'. Below this is a section 'Follow me on my social networks:' with links to GitHub, LinkedIn, Medium, and Instagram. At the bottom right, there is a section 'Popular repositories' showing 'EtecltuDsPw1' (Etec Martinho Di Ciero - Programação para Web 1 - de Software - 2018) with 1 star and 'dotnetConsoleDependencyInjection' (Exemplo de aplicação com injeção de dependência .net core) with 1 star.

Overview Repositories 52

NIZZOLA / README .md

Olá 👋 Bem vindos ao meu perfil!

- 👨‍🏫 Professor na Etec Martinho di Ciero
- 👥 Membro fundador da Comunidade
- 🔧 Atualmente trabalhando com .NET
- 🚀 Tenho estudado .NET, Angular, Ioni

Follow me on my social networks:

GitHub LinkedIn Medium Instagram

Popular repositories

EtecltuDsPw1

Etec Martinho Di Ciero - Programação para Web 1 - de Software - 2018

HTML ☆ 1

dotnetConsoleDependencyInjection

Exemplo de aplicação com injeção de dependência .net core

C# ☆ 1